CheapPhone Lab

Test Beyond the Flagship.

A developer testing & diagnostics platform built to uncover, prioritize and reproduce application failures under realistic Indian mobile hardware and telecom conditions.

HackFront India 2026 — Track T1.2: AI & Developer Tools

Target Track: Neo4j Graph Database Track | Prototype Demonstration

The Problem: The Flagship Fallacy in Indian App Development

01

Hardware Reality Gap

Flagship vs. Sub-₹8,000 Phones

• Developers test on 8GB-12GB RAM phones over 300Mbps broadband Wi-Fi.

• Over 400M Indian users use sub-₹8k phones with 1GB-2GB RAM (Android Go edition).

• Aggressive Low Memory Killers (LMK) kill app activities abruptly during transit.

02

Telecom & Storage Bottlenecks

2G/EDGE & Constrained Flash

• Commuter travel in tier-2/3 cities causes RTT latency spikes > 1500ms and 30%+ packet loss.

• Low remaining internal storage (< 500MB) causes SQLite databases to crash with SQLITE_FULL.

• Battery Saver governors throttle CPU clock down to 800MHz little cores, causing ANRs.

03

Silent User Abandonment

Catastrophic Churn

• Uncaught OOMs and frozen UIs cause users to uninstall apps immediately without sending bug reports.

• Duplicate form & order submissions occur because slow networks lag button debounce logic.

• Engineering teams lack tools to model and reproduce these stress realities locally.

The Solution: CheapPhone Lab Core Capabilities

⚡ Dual-Mode Testing

Mode A: Deterministic zero-device simulation engine. Mode B: Real ADB emulator & Logcat integration.

📱 Indian Budget Profiles

Pre-configured hardware profiles modeling 1GB-2GB RAM margins, Android Go editions, and tight flash storage.

🌐 Realistic Network Shaping

Presets for 2G/EDGE (128kbps), high latency (RTT > 1500ms), elevator disconnects, and packet jitter.

⚠️ 12 Failure Categories

Detects Out-of-Memory crashes, ANR looper stalls, socket timeouts, lost activity state, and double submissions.

📊 Transparent Impact Model

Quantifies affected users based on condition prevalence and target population with P0-P3 priority scoring.

🕸️ Neo4j Graph Explorer

Graph pattern matching across Application -> Feature <- Failure <- TestRun -> Condition -> UserSegment.

Project Workflow: From Configuration to Hotfix Recipe

STEP 1

Configure Test Lab

Select budget profile, network preset (2G/3G), storage margin, memory pressure, and battery saver.

STEP 2

Execute Scenario

Run deterministic simulation rules (instant) or launch controlled ADB Logcat on connected emulator.

STEP 3

Discover Failures

App isolates OOM stacktraces, ANRs, socket timeouts, and state loss across affected app features.

STEP 4

Impact Triage

Calculate estimated user reach: Population × Condition Prevalence × Failure Probability.

STEP 5

Neo4j Graph Analysis

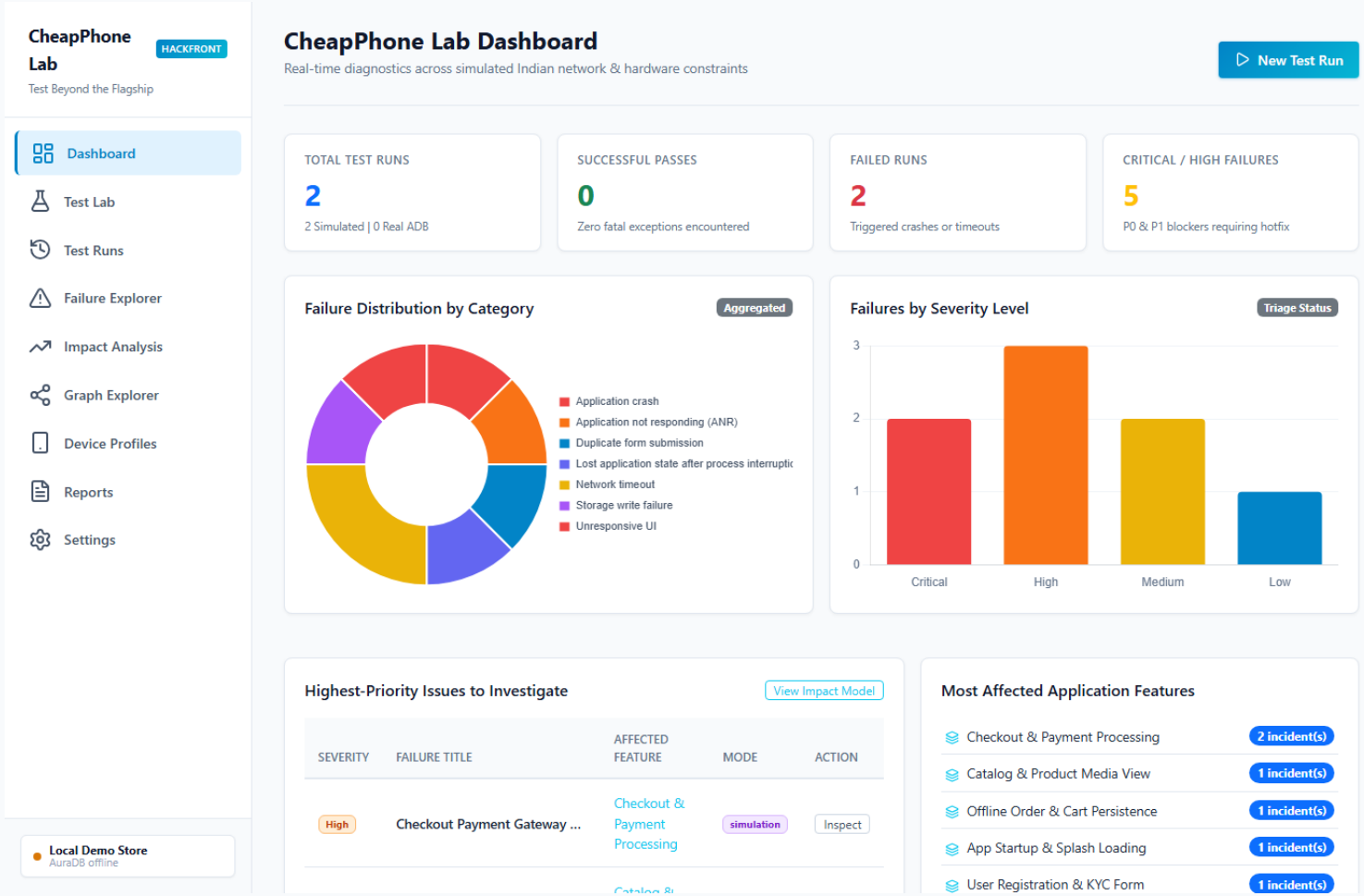
Traverse cross-run patterns to identify systemic hardware constraints causing correlated crashes.

STEP 6

Export Reproduction

Download actionable step-by-step reproduction recipes in Markdown and JSON for engineering sprint hotfixes.

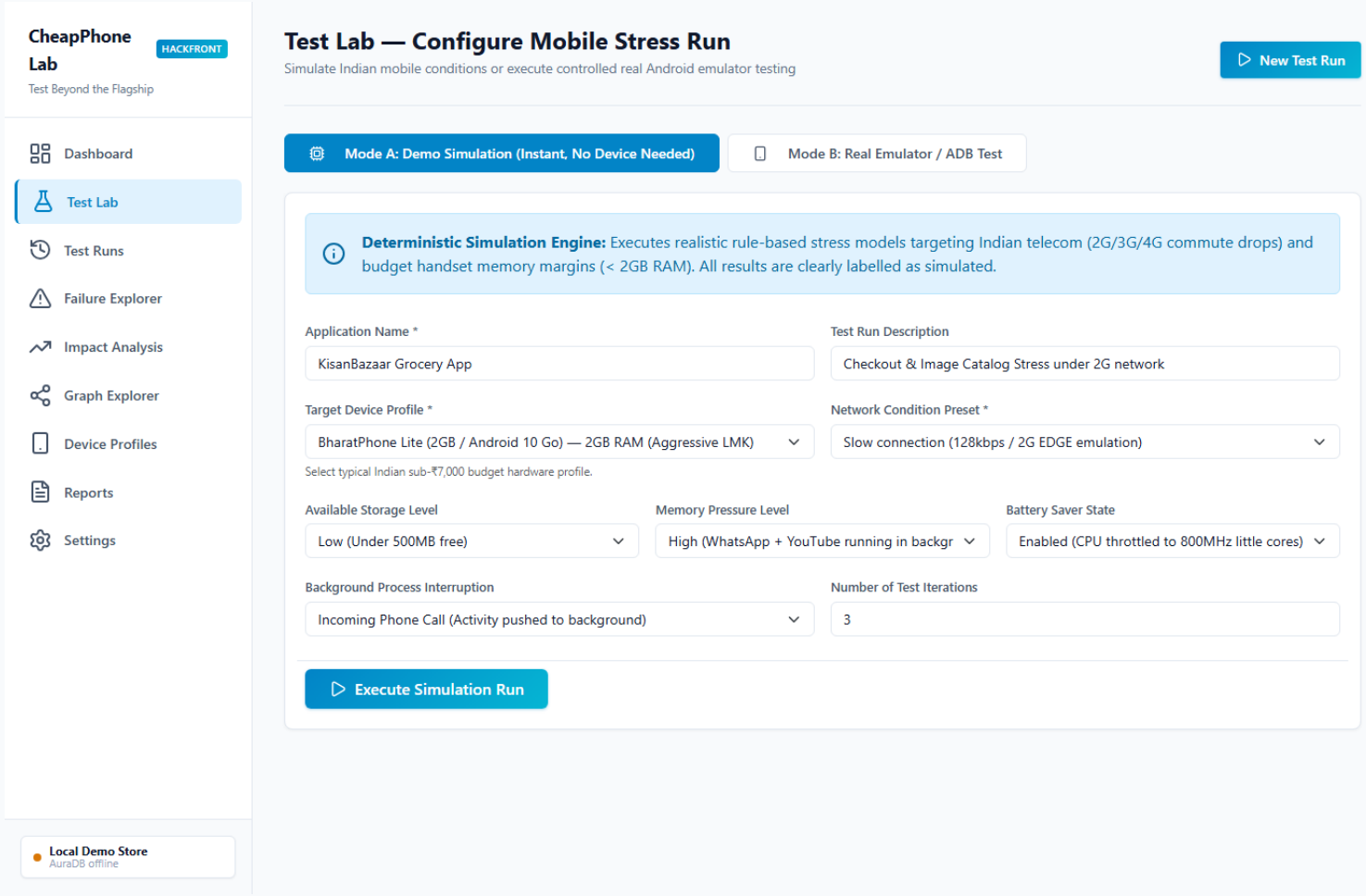
Prototype Walkthrough: Central Dashboard



Executive Diagnostics

- Live Run Metrics: Instant visibility into total test runs, pass/fail ratio, and critical P0/P1 blockers.
- Interactive Chart.js Visualizations: Failure distribution by category and severity triage breakdown.
- Priority Triage Queue: Direct highlights of top blocking failures and most vulnerable application modules.
- Recent Execution Activity: Real-time telemetry log tracking simulated and real ADB test outcomes.

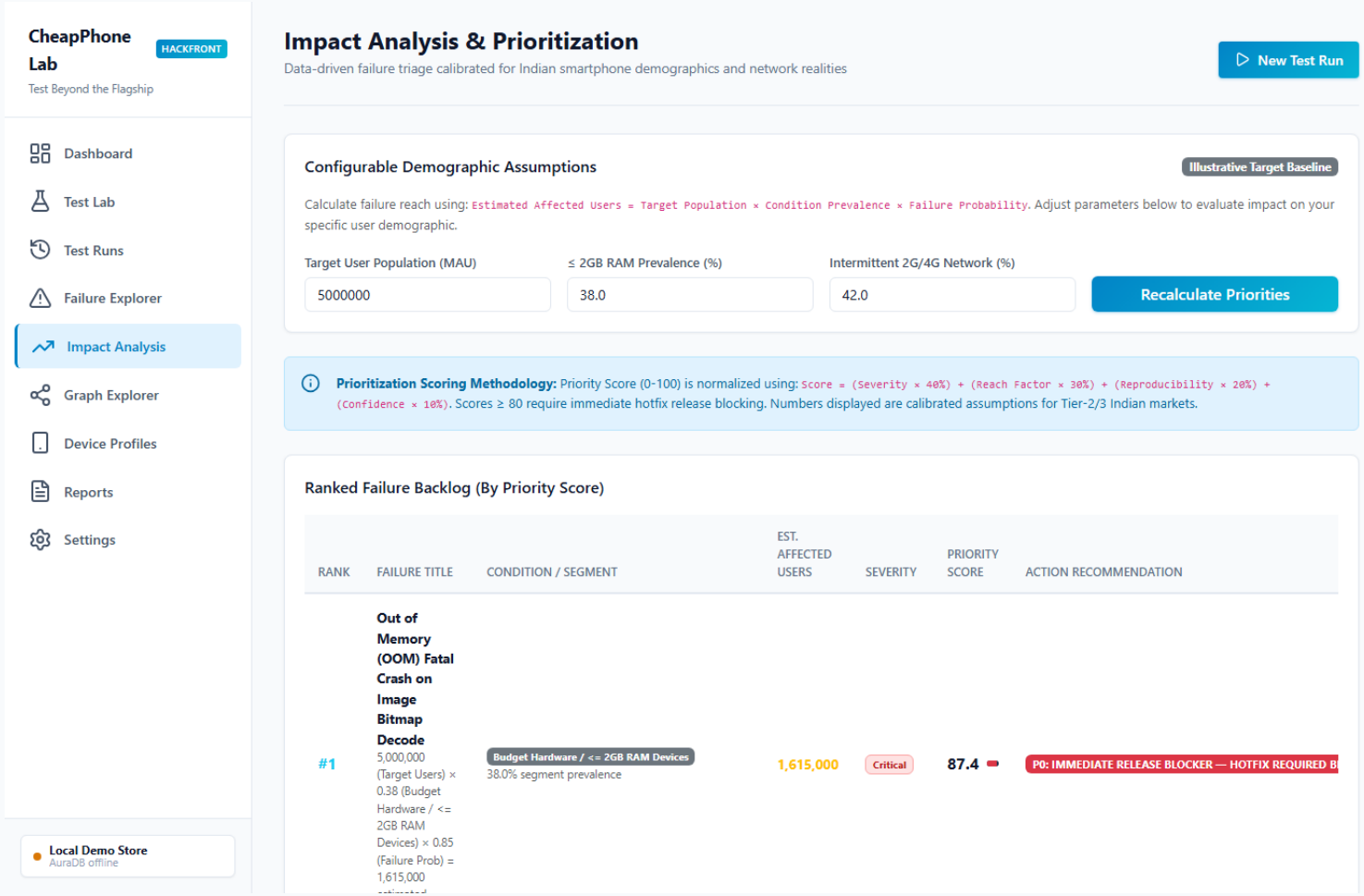
Prototype Walkthrough: Test Lab Configuration



Stress Simulation Engine

- Mode A (Zero-Device Simulation): Immediate deterministic stress modeling for rapid hackathon evaluations.
- Mode B (Real ADB / Emulator): Auto-detects connected emulators and retrieves actual device properties via getprop.
- Configurable Stress Factors: Adjust network latency (2G/EDGE), available storage (<500MB), RAM pressure, and battery saver.
- Activity Interruption: Models background phone calls and Android Go OS task evictions.

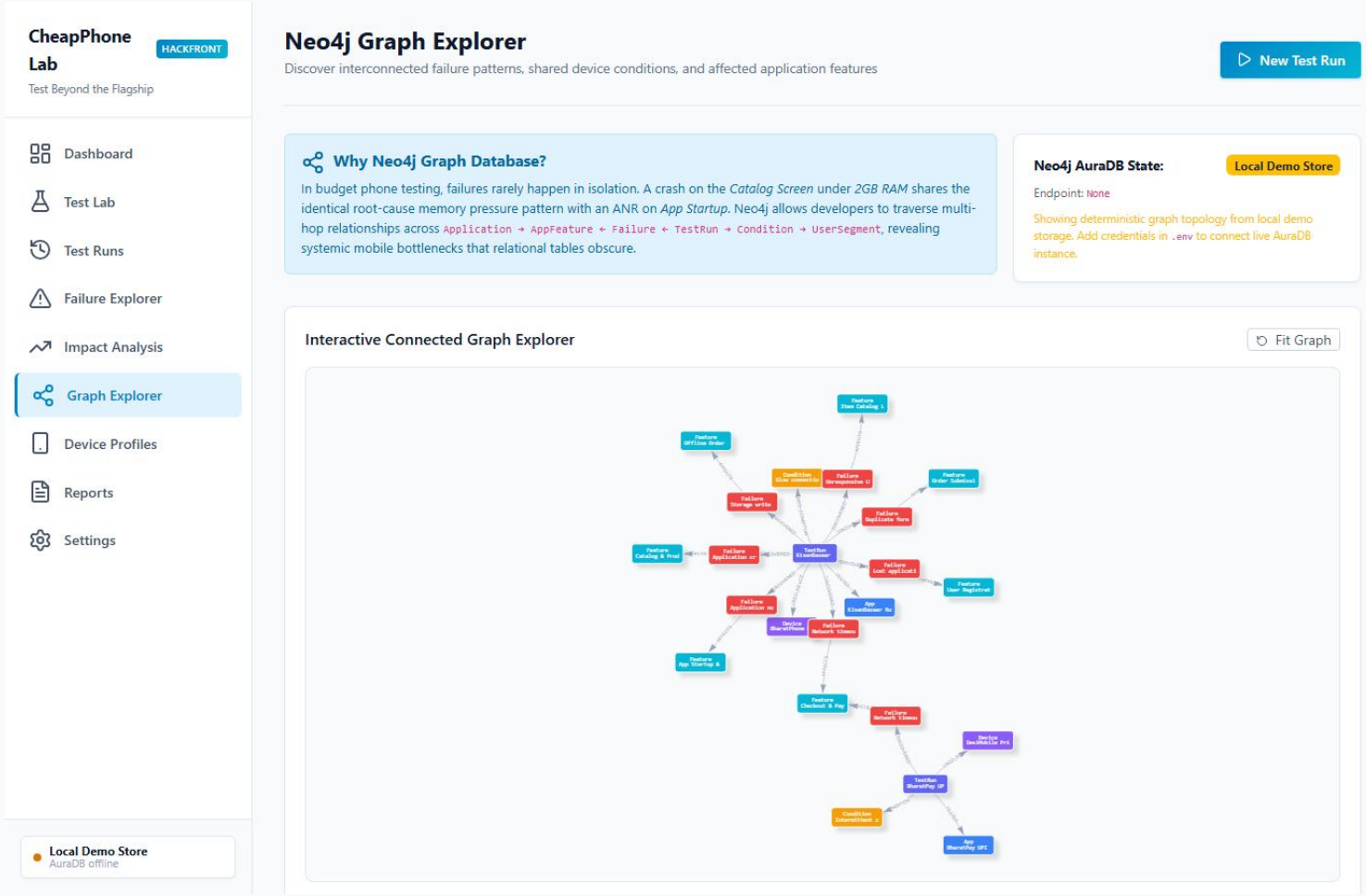
Prototype Walkthrough: Transparent Impact Analysis



Data-Driven Triage

- Transparent Mathematical Model: Affected = Target Pop × Condition Prevalence × Failure Probability.
- Configurable Demographics: Customize MAU size, budget hardware percentage, and intermittent network coverage.
- Normalized Priority Scoring (0-100): Severity (40%) + Reach (30%) + Repro (20%) + Confidence (10%).
- Actionable Classification: Categorizes failures into P0 Hotfix Blockers, P1 Sprint Priorities, and P2 Polish items.

Neo4j Graph Explorer: Cross-Run Pattern Matching



Graph Traversal Value

- Multi-Hop Schema: Connects Application -> AppFeature <- Failure <- TestRun -> Condition -> UserSegment.
- Systemic Root-Cause Discovery: Uncovers how disparate failures (e.g. OOM in Catalog & ANR on Startup) share identical stress nodes (2GB RAM).
- Interactive Vis-Network UI: Real-time visual graph canvas with drag, zoom, and neighborhood exploration.
- AuraDB Live / Fallback Store: Seamless Cypher execution with disconnected local storage fallback.

Technical Stack & Strict Architectural Adherence

Frontend Layer — HTML5, CSS3, Vanilla JS, Bootstrap 5

Clean light developer dashboard, responsive layouts, zero heavy node frameworks, Chart.js for charts, Vis-network for graph visualization.

Backend & Core — Python 3.11, Flask, python-dotenv

Modular architecture: core/test_runner.py, impact_analyzer.py, demo_scenarios.py, failure_detector.py with full REST API endpoints.

Graph Database — Neo4j AuraDB & Official Python Driver

Real parameterized Cypher MERGE queries, schema constraints, subgraph extraction, and graceful disconnected demo store fallback.

Diagnostics & Testing — ADB Subprocess, Logcat, pytest

Safe argument-list ADB runner (no shell injection), Logcat regex parsing for fatal crashes & ANRs, 5 automated pytest suites passing.

CheapPhone Lab: Test Beyond the Flagship

Empowering engineering teams to build software that thrives across India's next 500 million mobile users.

- ✓ Discovers hidden failure modes before they reach play store reviews and cause user churn.
- ✓ Provides reproducible, step-by-step engineering hotfix recipes with Logcat stack traces.
- ✓ Demonstrates real Neo4j graph traversal value in debugging complex mobile system constraints.
- ✓ Works out of the box with zero configuration for hackathon evaluations and developer CI/CD.